

SKILL_03

OUTPUT:

```
v_santosh@DESKTOP-E34M7RI:~$ mkdir Skill_03.c
v_santosh@DESKTOP-E34M7RI:~$ cd Skill_03.c
v_santosh@DESKTOP-E34M7RI:~/Skill_03.c$ nano Skill_03.c
v_santosh@DESKTOP-E34M7RI:~/Skill_03.c$ gcc Skill_03.c -o Skill_03
v_santosh@DESKTOP-E34M7RI:~/Skill_03.c$ ./Skill_03
Escape Sequence Demo
Hello   World
Line 1
Line 2
Quote: "Hello"
Backslash: \

Enter commands.
Type 'history' to display history.
Type 'prev' to recall previous command.
Type 'exit' to quit.

shell> exit

All dynamically allocated memory released.
v_santosh@DESKTOP-E34M7RI:~/Skill_03.c$ |
```

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 50

/* Node for command history */
typedef struct Node {
    char *command;
    struct Node *next;
} Node;

/* Add command to history */
void addHistory(Node **head, const char *cmd) {
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }

    newNode->command = (char *)malloc(strlen(cmd) + 1);

    if (newNode->command == NULL) {
        free(newNode);
        printf("Memory allocation failed!\n");
        return;
    }

    strcpy(newNode->command, cmd);
    newNode->next = *head;
    *head = newNode;
}

/* Display command history */
void displayHistory(Node *head) {
    int count = 1;

    printf("\nCommand History:\n");

    while (head != NULL) {
        printf("%d. %s\n", count++, head->command);
        head = head->next;
    }
}

/* Free linked list memory */
void freeHistory(Node *head) {
    Node *temp;

    while (head != NULL) {
        temp = head;
        head = head->next;

        free(temp->command);
        free(temp);
    }
}

```

```

        buffer = temp;

        /* Read remaining input */
        if (fgets(buffer + strlen(buffer),
                    size - strlen(buffer),
                    stdin) == NULL)
            break;
    }

    /* Exit */
    if (strcmp(buffer, "exit") == 0) {
        break;
    }

    /* Display history */
    if (strcmp(buffer, "history") == 0) {
        displayHistory(history);
        continue;
    }

    /* Recall previous command */
    if (strcmp(buffer, "prev") == 0) {
        if (history != NULL) {
            printf("Previous command: %s\n",
                    history->command);

            /* Update input buffer */
            strcpy(buffer, history->command);
        } else {
            printf("No previous command.\n");
        }
        continue;
    }

    /* Store command in history */
    if (strlen(buffer) > 0) {
        addHistory(&history, buffer);
    }
}

/* -----
5. Release Memory Correctly
----- */
free(buffer);
freeHistory(history);

printf("\nAll dynamically allocated memory released.\n");

return 0;
}

```

```

int main() {
    /* -----
    1. Apply Escape Sequences
    ----- */
    printf("Escape Sequence Demo\n");
    printf("Hello\tWorld\n");
    printf("Line 1\nLine 2\n");
    printf("Quote: \"Hello\"\n");
    printf("Backslash: \\n");

    /* -----
    2. Dynamically Allocate Input Buffer
    ----- */
    int size = INITIAL_SIZE;
    char *buffer = (char *)malloc(size);

    if (buffer == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    /* -----
    3. Command History
    ----- */
    Node *history = NULL;

    printf("\nEnter commands.\n");
    printf("Type 'history' to display history.\n");
    printf("Type 'prev' to recall previous command.\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {
        printf("shell> ");

        /* -----
        4. Update / Resize Input Buffer
        ----- */
        if (fgets(buffer, size, stdin) == NULL)
            break;

        /* Remove newline */
        buffer[strcspn(buffer, "\n")] = '\0';

        /* Check if buffer is too small */
        while (strlen(buffer) == size - 1 &&
            buffer[size - 2] != '\n') {

            size *= 2;

            char *temp = (char *)realloc(buffer, size);

            if (temp == NULL) {
                printf("Unable to resize buffer!\n");
                free(buffer);
                freeHistory(history);
                return 1;
            }
        }
    }
}

```